

Algoritmi e strutture dati - prova

Svolgimenti

Esercizio 1. (Punti 3) Si dimostri oppure si refuti ognuna delle seguenti inclusioni:

$$\begin{aligned}3^{n+2} &\in O(2^n) \\ 2^{n-2} &\in \Theta(3^n) \\ 2n^2 + 10n + 4 &\in \Omega(n^3)\end{aligned}$$

Soluzione 1. Poiché $3^{n+2} = 3^n \cdot 3^2$ ne segue che $3^{n+2} \in \Theta(3^n)$; d'altra parte:

$$\lim_{n \rightarrow \infty} \frac{2^n}{3^n} = \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0 \quad \text{poiché} \quad 0 < \frac{2}{3} < 1.$$

Ne segue che $2^n = o(3^n) = o(3^{n+2})$ e dunque $3^{n+2} \notin O(2^n)$.

Dato che $2^{n-2} = \frac{2^n}{4}$ chiaramente $2^{n-2} \in \Theta(2^n) \subset O(3^n)$. Tuttavia per quanto visto sopra $3^n \notin O(2^n)$ e dunque $2^n \notin \Omega(3^n)$. Se ne conclude che $2^{n-2} \notin \Theta(3^n)$.

Infine $2n^2 + 10n + 4 \in \Theta(n^2)$ essendo un polinomio di grado 2; ma $n^2 = o(n^3)$ e quindi $2n^2 + 10n + 4 \notin \Omega(n^3)$.

Esercizio 2. (Punti 6) Supponendo che la funzione $\text{FOO}(n)$ richieda tempo $O(\log n)$, si analizzi la complessità temporale in funzione di n del seguente algoritmo, ricavando la relazione di ricorrenza e calcolandone l' O -grande senza far ricorso al master theorem:

```
GREMLINS( $n$ )
  Pre:  $n > 1$  intero
  if  $n = 2$  then
    return 1
  else
    temp  $\leftarrow$  GREMLINS( $n - 1$ ) + FOO( $n$ )
    return temp
```

Soluzione 2. La relazione di ricorrenza di GREMLINS è:

$$T(n) = \begin{cases} O(1) & n = 0 \\ T(n-1) + \log n & n > 0 \end{cases}$$

Svolgendo si trova che:

$$T(n) = T(n-1) + \log n = T(n-2) + \log(n-1) + \log n = \dots = T(n-k) + \sum_{i=0}^{k-1} \log(n-i)$$

Dato che l'equazione vale per ogni $k \leq n$, ponendo $k = n$ abbiamo che:

$$T(n) = T(0) + \sum_{i=0}^{n-1} \log(n-i) \leq O(1) + \log(n!) = O(n \log n).$$

Esercizio 3. (Punti 6) Si consideri il seguente algoritmo:

```

CASPER( $A[0..n-1]$ )
  Pre:  $A$  array di interi
  for  $i \leftarrow 0$  to  $n-2$  do
    for  $j \leftarrow i+1$  to  $n-1$  do
      if  $A[i] = A[j]$  then
        return false
  return true

```

Si risponda succintamente alle seguenti domande:

1. Quando CASPER(A) ritorna **true**?
2. Qual è la sua complessità in termini di Θ ?
3. Sapreste indicare un algoritmo che risolva lo stesso problema in tempo asintoticamente migliore?

Soluzione 3.

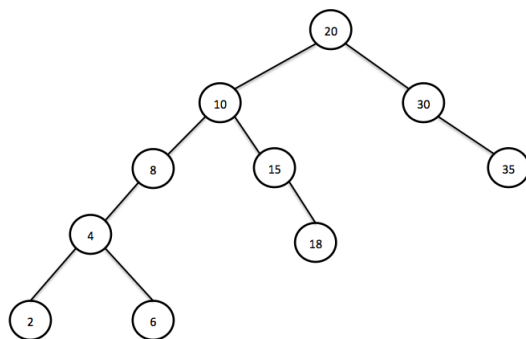
1. CASPER(A) ritorna **true** se per ogni $i, j \in 0..n-1$ si abbia che $A[i] \neq A[j]$.
2. La complessità di CASPER è dello stesso ordine del numero delle ripetizioni dell'**if**, ossia

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2).$$

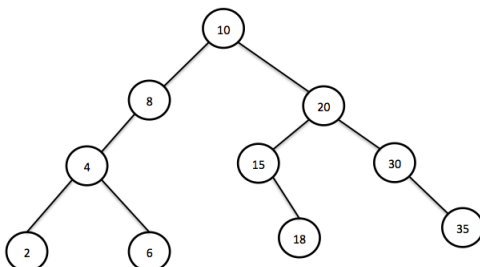
3. Se ordiniamo $A[0..n-1]$ con un algoritmo di costo $\Theta(n \log n)$, come MERGESORT o HEAPSORT, ogni elemento ripetuto nell'array originale sarà contiguo ad una delle sue ripetizioni, cosa che può verificarsi con un semplice algoritmo $\Theta(n)$. In questo modo si ottiene un algoritmo di costo $\Theta(n \log n) + \Theta(n) = \Theta(n \log n)$, asintoticamente migliore del quadratico CASPER.

Esercizio 4. (Punti 4) Si effettui l'inserimento delle seguenti chiavi in un albero binario di ricerca inizialmente vuoto: 20, 10, 30, 35, 8, 4, 2, 6, 15, 18, 35. Si effettui quindi una rotazione intorno alla radice in modo tale che l'albero diventi più bilanciato.

Soluzione 4. L'albero binario di ricerca risultante è (dove il 35 viene inserito una volta sola):



L'albero è sbilanciato perché mentre l'altezza del sottoalbero sinistro è 3, quella del destro è 1. Dopo un rotazione destra con centro sulla radice si ottiene:



in cui le altezze del sottoalbero sinistro e destro sono uguali e pari a 2.

Esercizio 5. (Punti 6) Sia dato un albero k -ario T , rappresentato con puntatori *child* e *sibling*, i cui nodi sono etichettati con interi. Calcolare in tempo $O(n)$, dove n rappresenta la cardinalità dell'albero, il massimo delle somme delle etichette sullo stesso ramo.

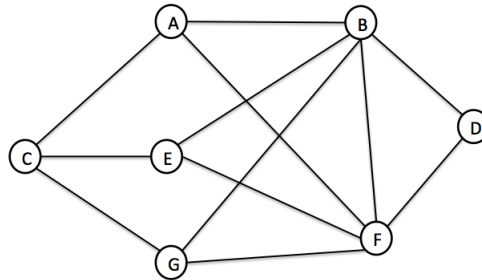
Soluzione 5. Nel caso in cui $T = nil$ il valore da ritornare è 0 ossia la sommatoria vuota; resta da trattare il caso in cui T sia non vuoto:

```

MAX-SUM-BRANCH( $T$ )
  if  $T.child = nil$  then
    return  $T.key$ 
  else
     $max\_sum \leftarrow \text{MAX-SUM-BRANCH}(T.child)$ .
     $C \leftarrow T.child.sibling$ 
    while  $C \neq nil$  do
       $sum \leftarrow \max(max\_sum, \text{MAX-SUM-BRANCH}(C))$ 
       $C \leftarrow C.sibling$ 
    return  $T.key + max\_sum$ 

```

Esercizio 6. (Punti 6) Un grafo $G = (V, E)$ si dice *bipartito* se esiste una partizione di V in due sottoinsiemi V_1 e V_2 tale che nessun arco $(u, v) \in E$ abbia i vertici u, v contenuti nella stessa parte, ossia per ogni $(u, v) \in E$ o $u \in V_1$ e $v \in V_2$, oppure $u \in V_2$ e $v \in V_1$. Decidere se il seguente grafo è bipartito oppure no, fornendo una partizione V_1, V_2 di V nel caso affermativo; spiegare perché nessuna partizione di V soddisfa la proprietà succitata nel caso negativo.

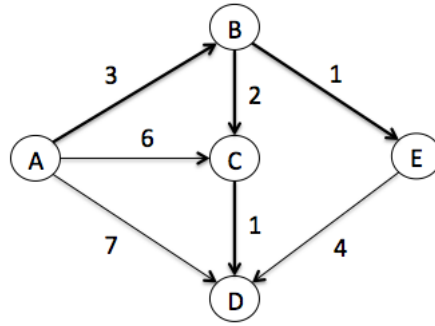


Soluzione 6. Il grafo non è bipartito perché contiene cicli di lunghezza dispari, come ad esempio BEF . Un grafo è bipartito se ogni suo sottografo lo è; d'altra parte il sottografo BEF non è bicoloreabile (o equivalentemente non è bipartito) poiché almeno due vertici adiacenti devono avere lo stesso colore.

Esercizio 7. (Punti 5) Si applichi l'algoritmo di Dijkstra al grafo riportato sotto con le liste di adiacenti a partire dal nodo A . Riportare il contenuto della coda di priorità prima di ogni estrazione.

$A : B(3), C(6), D(7)$
 $B : C(2), E(1)$
 $C : D(1)$
 $D :$
 $E : D(4)$

Soluzione 7. Il grafo, in cui gli archi di maggior spessore sono quelli che compongono i cammini minimi da A scelti dall'algoritmo di Dijkstra, risulta:



Quelli che seguono sono gli stati della coda (rappresentata come uno heap minimo in forma di array) prima dell'estrazione di ciascun vertice:

$(A, 0), (B, \infty), (D, \infty), (C, \infty), (E, \infty)$
 $(B, 3), (D, 7), (C, 6), (E, \infty)$
 $(E, 4), (D, 7), (C, 5)$
 $(C, 5), (D, 7)$
 $(D, 6)$